Com S // Cpr E // Math 5250

Numerical Analysis of High-Performance Computing

Instructor: Songting Luo

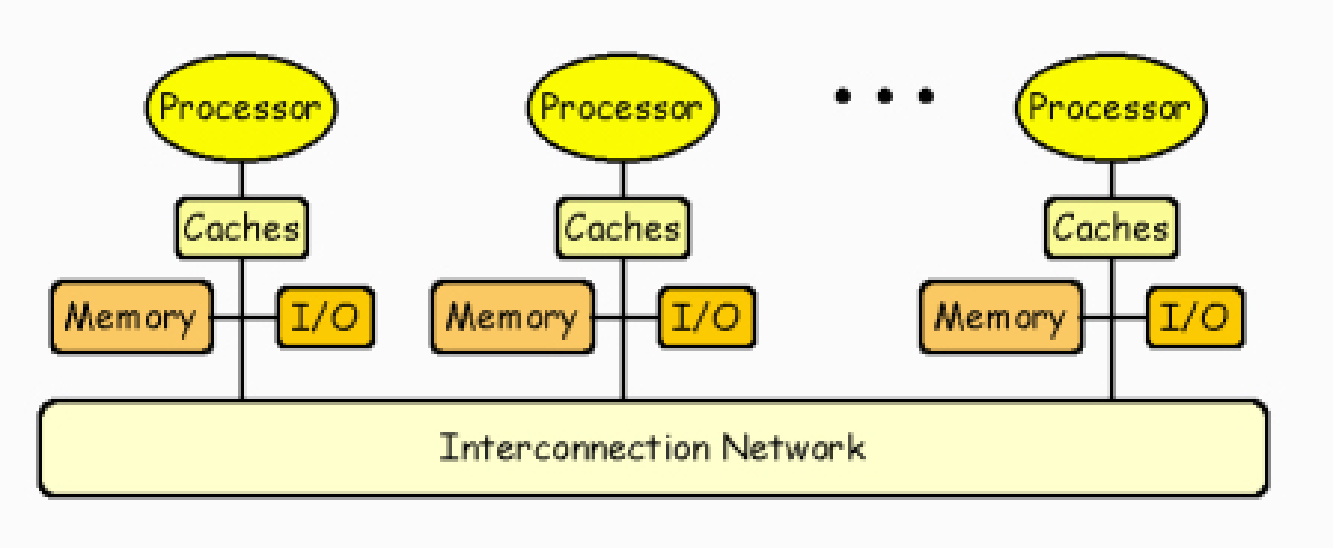
Lecture 21: Introduction to MPI

Outline

1. Distributed memory computing
2. Introduction to MPI
3. Computing the norm of a distributed vector
4. MPI implementation of the trapezoidal rule

Distributed memory computing

Parallel Processing: Distributed Memory



Distributed memory:

Each processor has its own memory and caches.

Transferring data between processors is slow.

Examples: clusters of computers, supercomputers.

Multi-processor computing

Some issues:

Generally more complicated to program than OpenMP on shared memory systems.

High cost of data communication between processes.

Want to maximize processing on local data relative to communication with other processes.

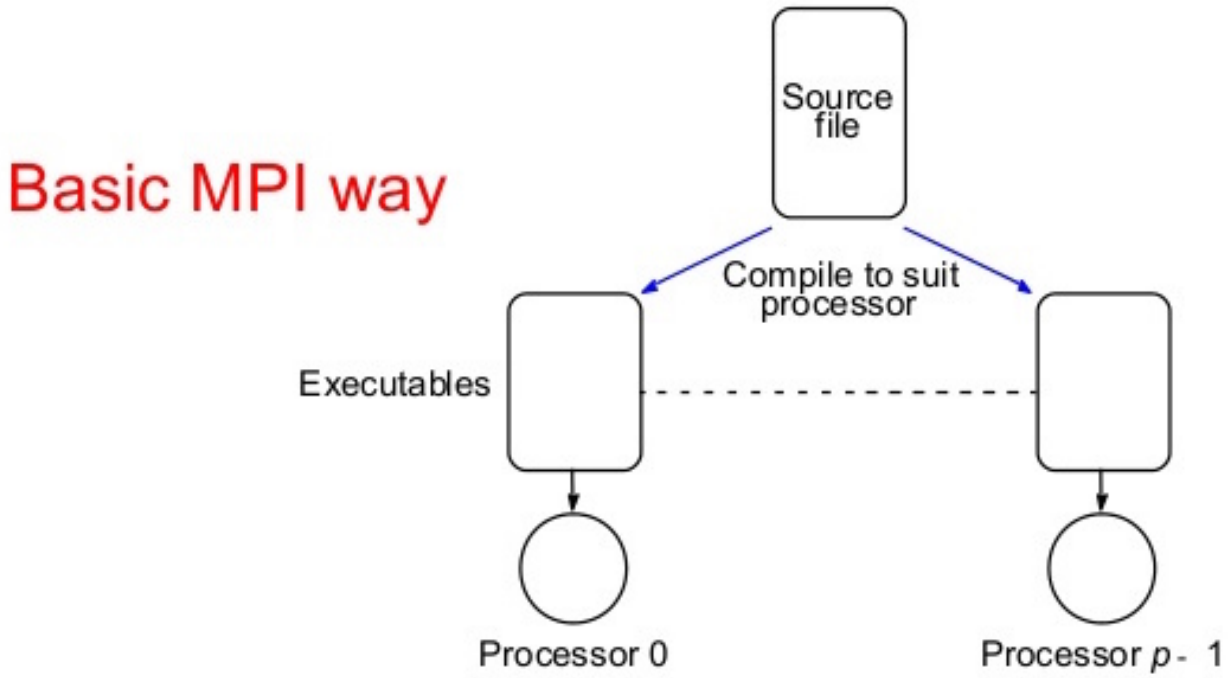
Often need to partition problem into subdomains

(e.g., domain decomposition for PDEs)

Generally requires coarse grain parallelism

Introduction to MPI

MPI: Message-Passing Interface



Message Passing:

SPMD model (Single Program, Multiple Data): all processors execute same program, but with different data.

Program manages memory by placing data in processes.

Data that must be shared is explicitly sent between processes.

MPI References

There are several implementations of MPI available.

A common choice is Open MPI:

`http://www.open-mpi.org/`

Another widely used implementation is MPICH from Argonne National Lab:

`http://www.mpich.org/`

Standard reference book:

W. Gropp, E. Lusk, A. Skjellum, *Using MPI*, Second Edition, MIT Press, 1999. (link)

See also: Bill Gropp's tutorial (link)

Hello world!

$ISUHPC/lectures/lecture21/codes/hello_mpi/main.c:

```c
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <mpi.h>

const int MAX_STRING = 100;

int main()
{
   char greeting[MAX_STRING];
   int comm_sz;
   int my_rank;

   MPI_Init(NULL,NULL);
   MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
   MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
```

Hello world!

$ISUHPC/lectures/lecture21/codes/hello_mpi/main.c:

```c
    //printf("Hello from Processor %2d of %2d\n",my_rank,
        comm_sz);


    if (my_rank != 0)
    {
       sprintf(greeting,"  Greetings from process %2i of %2i!",
               my_rank,comm_sz);
       MPI_Send(greeting,strlen(greeting)+1,MPI_CHAR,0,0,
                MPI_COMM_WORLD);
    }
    else
    {
       printf("  Greetings from process %2i of %2i!\n",
              my_rank,comm_sz);
       for (int j=1; j<comm_sz; j++)
       {
          MPI_Recv(greeting,MAX_STRING,MPI_CHAR,j,0,
                   MPI_COMM_WORLD,MPI_STATUS_IGNORE);
          printf("%s\n",greeting);
       }
    }

```

MPI Send and Receive

```
int MPI_Send(const void *buf,
             int count,
             MPI_Datatype datatype,
             int dest,
             int tag,
             MPI_Comm comm)

int MPI_Recv(void *buf,
             int count,
             MPI_Datatype datatype,
             int source,
             int tag,
             MPI_Comm comm,
             MPI_Status *status)
```

Hello world!

$ISUHPC/lectures/lecture21/codes/hello_mpi/Makefile:

```
CC = mpicc
FFLAGS = -O3 -Wall -std=c99
LFLAGS =
OBJECTS = main.o

main.exe:  $(OBJECTS)
        $(CC) $(LFLAGS) $(OBJECTS) -o main.exe

%.o: %.c
        $(CC) $(FFLAGS) -c $<

clean:
        rm -f $(OBJECTS) *.exe
```

Hello world!

```
$  make

$ mpirun -np 2 ./main.exe > output.data
```

$ISUHPC/lectures/lecture21/codes/hello_mpi/output.data:

```
   Greetings from process  0 of  2!
   Greetings from process  1 of  2!
```

Basic MPI program

```c
#include <mpi.h>

int main()
{
    int comm_sz; // number of processors
    int my_rank; // current processor

    MPI_Init(NULL,NULL);
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    .
    .
    .
    MPI_Finalize();
    return 0;
}
```

MPI Communicators

All communication takes place in groups of processes.

Communication takes place in some context.

A group and a context are combined in a communicator.

`MPI_COMM_WORLD` is a communicator provided by default that includes all processors.

`MPI_Comm_size(comm, &comm_sz)` returns the number of processors in communicator `comm`.

`MPI_Comm_rank(comm, &my_rank)` returns the rank of this processor in communicator `comm`.

MPI Library

The `#include<mpi.h>` library includes:

Functions such as

- `MPI_Init`
- `MPI_Comm_size`
- `MPI_Comm_rank`

Global variables such as

- `MPI_COMM_WORLD` : a communicator
- `MPI_INT` : used to specify type of data being sent
- `MPI_DOUBLE` : used to specify type of data being sent
- `MPI_SUM` : used to specify a type of reduction

Remember: C is case sensitive.

MPI functions

There are $\geq$ **125** MPI functions.

Can write many programs with these **8**:

- `MPI_Init` : initialize
- `MPI_Finalize` : finalize
- `MPI_Comm_size` : number of processors
- `MPI_Comm_rank` : rank of this processor
- `MPI_Send` : send a message
- `MPI_Recv` : receive a message
- `MPI_Bcast` : broadcast to other processors
- `MPI_Reduce` : reduction operation

Computing the norm of a distributed vector

Norm of a vector: main routine

$ISUHPC/lectures/lecture21/codes/norm_mpi/main.c:

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <mpi.h>
#include <math.h>
#include "vector.h"

int main(int argc, char* argv[])
{
   // Setup MPI code
   int comm_sz, my_rank;
   MPI_Init(NULL,NULL);
   MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
   MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

   // Grab the global N parameter
   // and set the local N parameter
   int N;
   void get_input(int argc, char* argv[],
                  const int my_rank,
                  const int comm_sz,
                  int* N);
   get_input(argc,argv,my_rank,comm_sz,&N);
```

Norm of a vector: main routine

$ISUHPC/lectures/lecture21/codes/norm_mpi/main.c:

```
    const int N_local = N/comm_sz;

    // Get time
    double time_start;
    if (my_rank==0)
    {  time_start = MPI_Wtime();  }

    // Create partial vector on current processor
    vector v_local = new_vector(N_local);
    for (int i=1; i<=N_local; i++)
    { vget(v_local,i) = sqrt((double)(i + my_rank*N_local)); }

    // Compute 2-norm squared
    double norm_squared;
    double norm_squared_local = pow(vget(v_local,1),2);
    for (int i=2; i<=N_local; i++)
    { norm_squared_local += pow(vget(v_local,i),2); }
```

Norm of a vector: main routine

$ISUHPC/lectures/lecture21/codes/norm_mpi/main.c:

```
    // Add local results to the global result on Processor 0
    if (my_rank != 0)
    {
       MPI_Send(&norm_squared_local,1,MPI_DOUBLE,0,0,
                MPI_COMM_WORLD);
    }
    else
    {
       norm_squared = norm_squared_local;
       for (int i=1; i<comm_sz; i++)
       {
          MPI_Recv(&norm_squared_local,1,MPI_DOUBLE,i,0,
                   MPI_COMM_WORLD,MPI_STATUS_IGNORE);
          norm_squared += norm_squared_local;
       }
    }
```

Norm of a vector: main routine

$ISUHPC/lectures/lecture21/codes/norm_mpi/main.c:

```
    // Print answer to screen
    if (my_rank==0)
    {
       double time_end = MPI_Wtime();
       double time_elapsed = time_end - time_start;
       printf(" NP = %2i, N = %i, norm_squared = %20.13e"
              ", norm = %20.13e\n",
              comm_sz,N,norm_squared,sqrt(norm_squared));
       printf("     Elapsed time = %20.13e\n",
              time_elapsed);
    }

    delete_vector(&v_local);

    // End program
    MPI_Finalize();
    return 0;
 }
```

Norm of a vector: main routine

$ISUHPC/lectures/lecture21/codes/norm_mpi/main.c:

```
void get_input(int argc, char* argv[],
               const int my_rank,
               const int comm_sz,
               int* N)
{
   void usage(const char *prog_name);
   if (my_rank==0)
   {
      if (argc != 2) { usage(argv[0]); }

      *N = strtol(argv[1], NULL, 10);
      if (*N<=0) { usage(argv[0]); }
      if (*N%comm_sz!=0) { usage(argv[0]); }

      for (int i=1; i<comm_sz; i++)
      {
         MPI_Send(N,1,MPI_INT,   i,0,MPI_COMM_WORLD);
      }
   }
```

Norm of a vector: main routine

$ISUHPC/lectures/lecture21/codes/norm_mpi/main.c:

```
    else
    {
       MPI_Recv(N,1,MPI_INT,0,0,MPI_COMM_WORLD,
                MPI_STATUS_IGNORE);
    }
}

void usage(const char *prog_name)
{
    fprintf(stderr,"usage: %s <N>\n",prog_name);
    fprintf(stderr,"   N should be positive\n");
    fprintf(stderr,"   N should be exactly divisible "
            "by the number of processors\n");
    exit(1);
}
```

Norm of a vector: Makefile

$ISUHPC/lectures/lecture21/codes/norm_mpi/Makefile:

```
CC = mpicc
FFLAGS = -O3 -Wall -std=c99
LFLAGS =
OBJECTS = main.o vector.o

main.exe:  $(OBJECTS)
        $(CC) $(LFLAGS) $(OBJECTS) -o main.exe

%.o: %.c
        $(CC) $(FFLAGS) -c $<

clean:
        rm -f $(OBJECTS) *.exe
```

Norm of a vector: output

```
$  make

$ mpirun -np 2 ./main.exe 1600000 > output.data
```

$ISUHPC/lectures/lecture21/codes/norm_mpi/output.data:

```
  NP =  2, N = 160000, norm_squared =  1.2800080000000e+10,
      norm =  1.1313743854269e+05
      Elapsed time =  5.9300000000000e-04
```

MPI implementation of the trapezoidal rule

Composite trapezoidal rule: main routine

$ISUHPC/lectures/lecture21/codes/trap_mpi/main.c:

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <mpi.h>
#include <math.h>

int main(int argc, char* argv[])
{
   // Setup MPI code
   int comm_sz, my_rank;
   MPI_Init(NULL,NULL);
   MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
   MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

   // Grab the global a, b, and N parameters
   double a,b,global_T,error;
   int N;
   void get_input(int argc, char* argv[],
                  const int my_rank,
                  const int comm_sz,
                  double* a,
                  double* b,
                  int* N);
```

Composite trapezoidal rule: main routine

$ISUHPC/lectures/lecture21/codes/trap_mpi/main.c:

```
    get_input(argc,argv,my_rank,comm_sz,&a,&b,&N);

    // Get time
    double time_start;
    if (my_rank==0)
    {  time_start = MPI_Wtime();  }

    // Run local trapezoidal rule
    const double h = (b-a)/((double)N);
    const int local_N = N/comm_sz;
    const double local_a = a + ((double)(my_rank*local_N))*h;
    const double local_b = local_a + ((double)local_N)*h;
    double CompTrap(const double a, const double b,
                    const int N, const double h);
    double local_T = CompTrap(local_a,local_b,local_N,h);
```

Composite trapezoidal rule: main routine

$ISUHPC/lectures/lecture21/codes/trap_mpi/main.c:

```
    // Add local results to the global result on Processor 0
    if (my_rank != 0)
    {
       MPI_Send(&local_T,1,MPI_DOUBLE,0,0,
                MPI_COMM_WORLD);
    }
    else
    {
       global_T = local_T;
       for (int i=1; i<comm_sz; i++)
       {
          MPI_Recv(&local_T,1,MPI_DOUBLE,i,0,
                   MPI_COMM_WORLD,MPI_STATUS_IGNORE);
          global_T += local_T;
       }
       const double Tex = exp(b)-exp(a);
       error = fabs(Tex - global_T)/fabs(Tex);
    }
```

Composite trapezoidal rule: main routine

$ISUHPC/lectures/lecture21/codes/trap_mpi/main.c:

```
    // Print answer to screen
    if (my_rank==0)
    {
       double time_end = MPI_Wtime();
       double time_elapsed = time_end - time_start;
       printf(" NP = %2i, N = %i, T = %20.13e, "
              "|T-Tex|/|Tex| = %20.13e\n",
              comm_sz,N,global_T,error);
       printf("     Elapsed time = %20.13e\n",
              time_elapsed);
    }

    // End program
    MPI_Finalize();
    return 0;
}
```

Composite trapezoidal rule: main routine

$ISUHPC/lectures/lecture21/codes/trap_mpi/main.c:

```
void get_input(int argc, char* argv[],
               const int my_rank,
               const int comm_sz,
               double* a,
               double* b,
               int* N)
{
   void usage(const char *prog_name);
   if (my_rank==0)
   {
      if (argc != 4) { usage(argv[0]); }

      *a = strtod(argv[1], NULL);
      *b = strtod(argv[2], NULL);
      *N = strtol(argv[3], NULL, 10);
      if (*N<=0) { usage(argv[0]); }
      if (*N%comm_sz!=0) { usage(argv[0]); }
```

Composite trapezoidal rule: main routine

$ISUHPC/lectures/lecture21/codes/trap_mpi/main.c:

```
        for (int i=1; i<comm_sz; i++)
        {
           MPI_Send(a,1,MPI_DOUBLE,i,0,MPI_COMM_WORLD);
           MPI_Send(b,1,MPI_DOUBLE,i,0,MPI_COMM_WORLD);
           MPI_Send(N,1,MPI_INT,   i,0,MPI_COMM_WORLD);
        }
     }
     else
     {
        MPI_Recv(a,1,MPI_DOUBLE,0,0,MPI_COMM_WORLD,
                 MPI_STATUS_IGNORE);
        MPI_Recv(b,1,MPI_DOUBLE,0,0,MPI_COMM_WORLD,
                 MPI_STATUS_IGNORE);
        MPI_Recv(N,1,MPI_INT,   0,0,MPI_COMM_WORLD,
                 MPI_STATUS_IGNORE);
     }
  }
```

Composite trapezoidal rule: main routine

$ISUHPC/lectures/lecture21/codes/trap_mpi/main.c:

```
void usage(const char *prog_name)
{
   fprintf(stderr,"usage: %s <a> <b> <N>\n",prog_name);
   fprintf(stderr,"   N should be positive\n");
   fprintf(stderr,"   N should be exactly divisible by "
           "the number of processors\n");
   exit(1);
}

double CompTrap(const double a, const double b,
                const int N, const double h)
{
   double func(const double x);

   double T = 0.5*(func(a)+func(b));
   for (int i=1; i<N; i++)
   {  T += func(a + ((double)i)*h);  }

   return h*T;
}
```

Composite trapezoidal rule: Makefile

$ISUHPC/lectures/lecture21/codes/trap_mpi/Makefile:

```
CC = mpicc
FFLAGS = -O3 -Wall -std=c99
LFLAGS =
OBJECTS = main.o

main.exe:  $(OBJECTS)
        $(CC) $(LFLAGS) $(OBJECTS) -o main.exe

%.o: %.c
        $(CC) $(FFLAGS) -c $<

clean:
        rm -f $(OBJECTS) *.exe
```

Composite trapezoidal rule: output

```
$  make

$ mpirun -np 2 ./main.exe 0 1 160000> output.data
```

$ISUHPC/lectures/lecture21/codes/trap_mpi/output.data:

```
 NP =  2, N = 160000, T =  1.7182818284646e+00, |T-Tex|/|Tex|
     =  3.2525879127612e-12
     Elapsed time =  5.6600000000000e-04
```

Lab Assignment: Pass a Baton

Create a program where a "token" (an integer) is passed sequentially from Rank 0, to Rank 1, to Rank 2, and eventually back to Rank 0.

- Start: Rank 0 initializes a token $= 1000$.
- The Pass: Rank 0 sends the token to Rank 1.
  - Rank 1 receives it, increments it by 1, and sends it to Rank 2.
  - This continues until the last rank (size -1) sends the token back to Rank 0.
- Completion: Rank 0 receives the final token and prints the value to verify it made a full trip.
- Logging: Every rank should print: "Rank X received token Y from Rank Z". And "Rand X sent token Y to Rank Z".

Submit source codes, screenshots and update Git.